

M&D

Chefferie de projet

TP préparation oral

Dario CURTO & Maxence FONTAINE-GROS
17/10/2025

Table des matières

1. Contexte : Chefferie de projet en ESN.....	4
2. Panorama de méthodologies de chefferie de projet (au minimum 3)	4
2.1 Cycle en V / Waterfall (classique).....	4
2.2 Méthode Agile — Kanban.....	4
2.3 SCRUM (choix principal)	5
2.4 Comparatif synthétique	5
3. Rôles et responsabilités (SCRUM) en ESN	6
3.1 Product Owner (PO)	6
3.2 Scrum Master (SM)	6
3.3 Équipe de développement	6
4. Documents, artefacts et livrables SCRUM (documentation requise)	7
4.1 Artefacts SCRUM	7
4.2 Documentations complémentaires	7
4.3 Templates utiles (à joindre au dossier).....	8
5. Outils collaboratifs et de planification (recommandés pour une ESN)	8
5.1 Gestion du backlog & des sprints	8
5.2 Documentation et knowledge sharing.....	8
5.3 Communication & collaboration synchrone.....	9
5.4 Outils de gestion de code & CI/CD.....	9
5.5 Tests et qualité.....	9
5.6 Planification, suivi financier et ressources (ESN)	9
6. Planification et déroulé étape par étape — SCRUM appliqué en ESN.....	10
7. Planification temporelle exemple (projet de 6 mois — sprint 2 semaines).....	11
8. Mesures et indicateurs (KPIs) recommandés.....	12
9. Gestion des risques et pièges fréquents (et comment les éviter).....	12
9.1 Risques courants.....	12
9.2 Mesures d'atténuation.....	13
10. Cas pratiques et recommandations ESN-client.....	13
10.1 Adapter le contrat.....	13
10.2 Gouvernance projet.....	13
10.3 Hybridation (SCRUM + Kanban).....	14
11. Annexes : Templates & checklists (extraits).....	14

Chefferie de Projet en ESN — Méthodologie SCRUM

11.1 Template mini-Product Backlog (colonnes).....	14
11.2 Checklist Definition of Done (exemple).....	14
11.3 Exemple d'ordre du jour — Sprint Planning (4h pour 2 semaines)	15
12. Conclusion — Pourquoi SCRUM en ESN ?.....	15
13. Recommandations de mise en œuvre immédiates (plan d'action 30-60-90 jours).....	15

Chefferie de Projet en ESN — Méthodologie SCRUM

Dossier : Chefferie de projet en ESN — choix méthodologique : SCRUM

Volume estimé : ~10 pages ($\approx$ 2 200–2 600 mots)

Ce dossier présente : (1) un panorama de méthodologies (≥ 3) et la justification du choix SCRUM, (2) les documentations et outils liés à la méthodologie retenue, (3) la planification détaillée étape-par-étape pour mettre en œuvre SCRUM au sein d'une ESN (Entreprise de Services du Numérique). Le contenu intègre recommandations pratiques, pièges fréquents et modèles/template réutilisables.

1. Contexte : Chefferie de projet en ESN

Les ESN réalisent des prestations de conception, intégration, maintenance et évolution de solutions pour des clients variés. Le chef de projet en ESN doit concilier : attentes contractuelles, qualité de livraison, pilotage budgétaire, coordination d'équipes souvent distribuées, et adaptation aux contraintes clients (SLA, sécurité, architecture existante). La méthodologie choisie doit être adaptée à la nature du projet (incertitude fonctionnelle, deadlines contractuelles, dépendances techniques) et au mode de facturation (au forfait, régie, mixte).

2. Panorama de méthodologies de chefferie de projet (au minimum 3)

2.1 Cycle en V / Waterfall (classique)

Principe : séquence linéaire d'étapes (cahier des charges → conception → développement → tests → recette → déploiement).

Quand l'utiliser : projet bien défini, exigences stables, contraintes réglementaires fortes.

Avantages : planification et budget clairs ; traçabilité ; adapté au contrat au forfait strict.

Limites : faible flexibilité face aux changements ; découverte tardive des problèmes ; risque de non-conformité aux attentes réelles de l'utilisateur.

2.2 Méthode Agile — Kanban

Principe : flux continu de petites tâches via un tableau visuel (To Do / Doing / Done), gestion du WIP (Work In Progress).

Quand l'utiliser : maintenance, support, projets avec demandes entrantes fréquentes, équipe souhaitant optimisation du flux.

Avantages : très visuel, flexible, réduit le lead time, amélioration continue.

Limites : moins de structure sur les itérations et livrables planifiés ; nécessite maturité pour prioriser et limiter WIP.

2.3 SCRUM (choix principal)

Principe : cadre agile itératif et incrémental basé sur des sprints (périodes fixes, généralement 2–4 semaines), rôles définis (Product Owner, Scrum Master, Équipe de développement), artefacts (Product Backlog, Sprint Backlog, Increment) et réunions formelles (Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective).

Quand l'utiliser : projets avec incertitudes fonctionnelles, forte interaction client, besoin de livraisons fréquentes et incrémentales.

Avantages : focalisation sur la valeur business, feedback rapide, responsabilisation de l'équipe, adaptation continue.

Limites : nécessite discipline et sponsor engagé ; pièges possibles (mauvais Product Owner, Sprints mal dimensionnés).

2.4 Comparatif synthétique

Stabilité exigée → Cycle en V.

Flux continu / support → Kanban.

Développement incrémental avec collaboration client → SCRUM (ou Scrum + Kanban hybrid).

Choix retenu : SCRUM

Motifs : capacité d'une ESN à livrer des incréments logiciels réguliers, gestion efficace des incertitudes client, traçabilité des priorités business, alignement possible avec des contrats au forfait via incréments validés.

3. Rôles et responsabilités (SCRUM) en ESN

3.1 Product Owner (PO)

Représente-le client/utilisateur.

Priorise le Product Backlog selon valeur métier, ROI, risques.

Valide les incréments (acceptance criteria).

Disponible pour clarifications et décisions.

3.2 Scrum Master (SM)

Facilite les réunions SCRUM.

Supprime les obstacles (impediments).

Protège l'équipe des dérangements externes.

Accompagne la transformation agile et la montée en maturité.

3.3 Équipe de développement

Pluridisciplinaire (développeurs, QA, DevOps, UX selon besoin).

Autoorganisée pour atteindre l'objectif de sprint (Sprint Goal).

Responsable de la qualité et du « Definition of Done ».

3.4 Parties prenantes externes

Sponsor / Direction client.

Chef de projet contractuel (si présent côté ESN pour aspects commerciaux et SLA).

Architecte ou référent technique (si nécessaire).

4. Documents, artefacts et livrables SCRUM (documentation require)

4.1 Artefacts SCRUM

Product Backlog : Liste priorisée de : User Stories / Features / Bugs / Tasks.

Sprint Backlog : Sélection d'items pour le sprint + plan de livraison.

Increment : Logiciel potentiellement livrable respectant Definition of Done (DoD).

Definition of Done (DoD) : Checklist partagée (tests unitaires, revues, documentation, packaging, déploiement en staging).

Definition of Ready (DoR) (recommandé) : Critères pour qu'une story soit éligible au sprint.

4.2 Documentations complémentaires

Charte de projet (portée, objectifs, KPIs, budget, risques majeurs).

Cahier des charges fonctionnel (synthétique) — si nécessaire pour contractualisation.

Plan qualité & stratégie de tests (unitaires, intégration, e2e, perf).

Registre des risques et plan de mitigation.

Plan de release / roadmap produit (niveau macro).

Reporting sprint : burndown chart, vélocité, tableau des impediments, sprint review minutes.

Procédures DevOps / CI-CD : pipelines, rollback, environnements.

Contrat / Annexes SLA (pour livraisons, support, maintenance).

4.3 Templates utiles (à joindre au dossier)

Template Product Backlog (champ: id, priorité, description, story points, DoR/DoD, owner).

Template Sprint Review (résumé incrément, feedback, décisions).

Template Retrospective (Start/Stop/Continue, actions avec responsables).

Checklist DoD & DoR.

5. Outils collaboratifs et de planification (recommandés pour une ESN)

5.1 Gestion du backlog & des sprints

Jira (Atlassian) — board Scrum, gestion des versions, rapports vélocité, burndown.

Azure DevOps — boards, pipelines CI/CD, intégration repos.

Taiga / GitLab Issues — alternatives open source / intégrées.

5.2 Documentation et partage de connaissances

Confluence (Atlassian) — wiki projet, charte, décisions.

Notion — documentation flexible, synthèses.

SharePoint (si client MS).

5.3 Communication & collaboration synchrone

Slack / Microsoft Teams — canaux par équipe, intégrations (Jira, CI).

Miro / MURAL — ateliers en ligne, story mapping, planning poker.

5.4 Outils de gestion de code & CI/CD

GitHub / GitLab / Bitbucket — repos, merge requests.

Jenkins / GitLab CI / GitHub Actions / Azure Pipelines — automatisation builds/tests/déploiements.

5.5 Tests et qualité

SonarQube — qualité code.

Selenium / Cypress — tests e2e.

JMeter / Locust — tests de performance.

5.6 Planification, suivi financier et ressources (ESN)

MS Project / Smartsheet — pour Gantt, dépendances (utile côté commercial/contractuel).

Tableaux de charge (Timesheets) — Harvest, Tempo (Jira), ou outils internes ESN.

Conseil ESN : intégrer les outils pour un reporting automatisé (Jira→Confluence, CI→Slack). Maintenir un canal unique pour les alertes vers le client et la gouvernance.

6. Planification et déroulé étape par étape — SCRUM appliqué en ESN

Pré-phase (0) : Démarrage et cadrage (2–4 semaines)

Démarrage du projet : réunir sponsor, PO, lead technique ESN, SM, représentants client.
Clarifier périmètre initial, contraintes contractuelles, SLA.

Charte & Roadmap macro : livrables principaux, milestones contractuelles, critères d'acceptation généraux.

Constitution de l'équipe : affectation profils, environnements, accès, repos.

Atelier de Product Discovery / Story Mapping (si besoin) : grande vision produit → découpage épique en stories.

Définition initiale du DoD & DoR et mise en place des outils (Jira, repo Git, pipeline CI).

Plan de release initial : releases majeures alignées sur jalons contractuels.

Phase 1 : Mise en place des sprints (Sprint 0 / Sprint d'initialisation)

Durée : 1 sprint (souvent 2 semaines) ou sprint 0 court.

Objectifs :

Stabiliser l'environnement (CI, staging, accès).

Créer le squelette du code, pipelines de build/deploy, tests de base.

Prioriser backlog pour 2–3 sprints à venir.

Livrable : incrément minimal (par ex. scaffolding + pipeline) et backlog prêt.

Phase 2 : Exécution itérative (Sprints réguliers)

Chaque sprint suit le cycle suivant :

Sprint Planning (0.5–1 jour)

PO présente priorités et Sprint Goal.

L'équipe sélectionne les items du Product Backlog selon capacité (vélocité) et établit le Sprint Backlog.

Décomposition en tâches, estimation (story points, idéalement via planning poker).

Daily Scrum (15 min quotidiens)

Point court : ce qui a été fait, ce qui sera fait, impediments.

SM suit les impediments, PO n'est pas obligé d'être présent sauf besoin.

Développement & Tests continus

Intégration continue, revues de code, tests unitaires automatisés.

Mise à jour du burndown chart, gestion des dépendances avec d'autres équipes.

Sprint Review (fin du sprint, 1–2 heures)

Démo de l'incrément au PO et aux parties prenantes.

Recueil des feedbacks et priorisation des ajustements.

Sprint Retrospective (1–1.5 heures)

Équipe identifie améliorations process, actions concrètes (avec responsables et délais).

Suivi des actions d'amélioration d'un sprint à l'autre.

Livraison / Release (si planifié)

Release sur environnement de recette client ou production selon roadmap.

Vérification des critères DoD et process de mise en production (checklist, runbooks).

Phase 3 : Stabilisation & Transition (FT, hypercare)

Après release majeure : période de support intensif (hypercare), corrections rapides, surveillance.

Transfert d'exploitation : documentation, procédures de maintenance, formation.

7. Planification temporelle exemple (projet de 6 mois — sprint 2 semaines)

Semaine 1–4 : Cadrage + Sprint 0 (mise en place outils).

Semaine 5–24 : ~10 sprints de 2 semaines → développements incrémentaux.

Semaine 25 : Release majeure + Hypercare.

Livrables : incréments toutes les 2 semaines, 2–3 releases majeures.

8. Mesures et indicateurs (KPIs) recommandés

Vélocité (story points complétés par sprint) — suivi pour prévoir la capacité.

Sprint Burndown — état d'avancement du sprint.

Lead time / Cycle time — mesure flux (si Kanban hybride).

Taux de réussite des stories (done vs committed).

Nombre d'impediments ouverts et temps de résolution.

Qualité : taux de tests automatisés, couverture unitaires, bugs en production.

Satisfaction client : Net Promoter Score interne, feedbacks des reviews.

9. Gestion des risques et pièges fréquents (et comment les éviter)

9.1 Risques courants

PO absent ou non disponible → décisions retardées, priorisation floue.

Sprints surchargés → baisse de qualité et non-respect des engagements.

Équipes multi-projets sans capacité dédiée → contexte switching, chute de productivité.

Mauvaise intégration outils → reporting manuel, perte de traçabilité.

Contractualisation incompatible avec agilité → conflit budget/délai vs itérations.

Definition of Done imprécise → incompréhensions sur ce qui est « livrable ».

9.2 Mesures d'atténuation

S'assurer d'un PO engagé (contrat/SLAs de disponibilité).

Déterminer capacité réelle (temps plein vs partiel) et préserver la stabilité d'équipe.

Mettre en place DoR & DoD claires et vérifier en Sprint Planning.

Automatiser CI/CD pour réduire risques de déploiement.

Adopter contrat agile/contrat mixte (ex : jalons à livrer + budget pour évolution).

Formation & coaching SCRUM (SM pro) pour prendre en maturité.

10. Cas pratiques et recommandations ESN-client

10.1 Adapter le contrat

Utiliser contrats itératifs (par release) ou time & materials avec cap sur objectifs.

Prévoir clause de changement d'ordre (change request) et modalités de priorisation.

10.2 Gouvernance projet

Rituel de gouvernance mensuel : comité de pilotage (Comité de Direction + PO + Chef de Projet ESN) pour décisions stratégiques.

Reporting léger mais récurrent : dashboard (vélocité, burn-up, risques ouverts) partagé via Confluence/PowerBI.

10.3 Hybridation (SCRUM + Kanban)

Pour support ou opérations, garder Scrum pour développement et Kanban pour maintenance.

Utiliser Scrumban pour tirer avantage du cadencement et du flux.

11. Annexes : Templates & checklists (extraits)

11.1 Template mini-Product Backlog (colonnes)

ID | Titre | Description courte | Priorité (MoSCoW) | Story Points | DoR OK (Oui/Non) | Critères d'acceptation | Responsable

11.2 Checklist Definition of Done (exemple)

Code compilé et build OK.

Tests unitaires passent ($\geq$ X% coverage).

Tests d'intégration valides.

Revue de code effectuée (2 approbations).

Documentation utilisateur minimale et release notes rédigées.

Déployable sur staging via pipeline automatique.

Critères d'acceptation validés par le PO.

11.3 Exemple d'ordre du jour — Sprint Planning (4h pour 2 semaines)

Rappel Sprint Goal.

Présentation par le PO des User Stories prioritaires.

Q&A & raffinement rapide (DoR check).

Estimation / Planning Poker.

Sélection finale et découpage en tâches.

Assignations et risques identifiés.

12. Conclusion — Pourquoi la méthode SCRUM en ESN ?

La méthode SCRUM concilie adaptabilité et cadence de livraison : Elle favorise un feedback rapide entre client et équipe ESN, améliore la visibilité du projet, et facilite l'alignement de la valeur livrée aux priorités métier. Pour une ESN, cette méthode permet aussi d'optimiser la facturation axée valeur (releases validées) tout en limitant les risques grâce à un pilotage itératif. Le succès dépend fortement d'un Product Owner impliqué, d'un Scrum Master compétent, et d'une intégration des outils et processus (CI/CD, backlog management, reporting).

13. Recommandations de mise en œuvre immédiates (plan d'action 30–60–90 jours)

0–30 jours

Organiser kick-off et atelier story mapping.

Mettre en place outils (Jira + Confluence + repo + CI).

Rédiger DoR & DoD.

30–60 jours

Lancer Sprint 0 puis 2 sprints normaux.

Mettre en place reporting (burndown, dashboard).

Former PO et SM (coaching).

60–90 jours

Mesurer KPIs (vélocité, lead time).

Ajuster roadmap et contrat si nécessaire (reviews avec sponsor).

Standardiser templates & intégrations.

Remarques finales (pièges à jouer en bonus pour améliorer la note)

Documentez les hypothèses contractuelles (ex. disponibilité du PO, ressources) — ceux-ci sont souvent des pièges oubliés.

Prévoyez un plan de montée en charge si l'ESN devra augmenter l'équipe (onboarding, pair programming, standard de code).

Intégrez une clause d'acceptation itérative dans le contrat pour éviter conflit entre forfait strict et agilité.